UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

A

AIP Project: Report

On

Farm Irrigation System using Minimum Spanning Tree

Submitted for partial fulfillment of requirement for the award of

Degree of

Master of Computer Applications

Chandigarh University, Mohali

Submitted By:	Submitted To:
Tushar Sharma 26MCC20073 MCA (Cloud Computing and DevOps)	Asst. Prof. Shubham Sharma

Contents

S. No.	Section Title	Page No.
1.	Acknowledgement	1
2.	Introduction	1
3.	Objectives	1-2
4.	Technologies Used	2
5.	System Requirements	2-3
6.	Problem Statement and Proposed System	3-4
7.	Concepts Used	4-6
8.	System Design and Program Modules	7-8
9.	Code Implementation	8-12
10.	Sample Input and Output	12-15
11.	Input Validation	15-16
12.	Features and Functionalities	16-17
13.	Future Enhancements	17
14.	Conclusion	17
15.	References	18

1. Acknowledgement

I would like to express my sincere thanks to my project guide, Asst. Prof. Shubham Sharma, for the constant guidance, patient support and valuable suggestions given during this project. The clear explanations and encouragement helped me to understand the topic properly and to complete this report on time.

I am also thankful to the University Institute of Computing, Chandigarh University, Mohali, for giving me the chance to work on this project and for providing the facilities needed to complete it. I extend my thanks to all the faculty members of the department for their support.

Finally, I thank my family and friends for their encouragement and motivation. Without their help this work would not have been possible.

Tushar Sharma
26MCC20073
MCA (Cloud Computing and DevOps)

2. Introduction

Water is the most important need of a farm. A farm usually has many plots (small pieces of land), and all of them must get water from one source such as a well, a tank or a canal. To carry the water, pipelines have to be laid from the source to the plots, and a pipeline can also join one plot to another plot.

Laying pipes costs money. The farmer must pay for the pipe, for digging and for labour. If pipes are laid without planning, the farmer may spend much more than needed. So the farmer wants to know: which pipelines should be laid so that every plot gets water and the total cost is as small as possible?

This question is a very well known problem in computer science. It is solved with a Minimum Spanning Tree (MST). In this project the farm is treated as a graph: the water source and the plots are points, the possible pipelines are lines joining the points, and the cost of each pipeline is written on the line. The program is written in C++ and uses Kruskal's algorithm to find the cheapest set of pipelines.

Only simple concepts are used in the program: arrays, structures, functions, loops and conditions. All the functions are written by the user (me) and no ready-made library functions for sorting or graphs are used, so every step can be explained easily.

3. Objectives

The main objective of this project is to find the cheapest way of connecting all plots of a farm to a water source. The other objectives are:

- To understand the idea of a graph, a spanning tree and a minimum spanning tree.
- To apply Kruskal's algorithm to a real life problem, i.e. farm irrigation.
- To implement the algorithm in C++ using simple, user-defined functions only.
- To reduce the total pipe cost and avoid wasting pipes and money.
- To make sure that every plot is connected to the water source and no useless loop of pipes is created.
- To check the input given by the user so that the program does not crash or give wrong answers.
- To prepare a project that is easy to understand and easy to explain.

4. Technologies Used

The following technologies and tools were used to build the project and to prepare this report.

Technology / Tool	Use in the Project
C++ language	Main programming language used to write the whole program.
iostream header	The only header file used. It gives cin and cout for input and output.
Structure (struct)	Used to store one pipeline (two end points and its cost) as a single unit.
Arrays	Used to store all pipelines, the selected pipelines and the group of each point.
Kruskal's algorithm	Greedy method used to find the minimum spanning tree.
Selection sort	User-written sorting method to arrange pipelines from lowest cost to highest cost.
Union-Find (simple groups)	User-written functions to detect whether a pipeline would make a loop.
g++ compiler / Code::Blocks / Dev-C++ / VS Code	To compile and run the C++ program.
Python (Matplotlib)	Used only to draw the figures given in this report.
Microsoft Word	Used to prepare and format this report.

5. System Requirements

The program is very small, so it can run on almost any computer that has a C++ compiler.

5.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or equivalent (or higher)
RAM	2 GB minimum (4 GB recommended)
Hard disk	500 MB free space (mostly needed for the compiler / IDE)
Input device	Keyboard
Output device	Monitor

5.2 Software Requirements

Component	Requirement
Operating system	Windows 10 / 11, Linux or macOS
Compiler	g++ (GCC) or any C++ compiler that supports C++11 or newer
Editor / IDE	Code::Blocks, Dev-C++, Visual Studio Code or any text editor
Word processor	Microsoft Word (for viewing this report)

6. Problem Statement and Proposed System

6.1 Problem Statement

A farm has one water source and several plots. Some pipelines between the source and the plots, and between the plots themselves, are possible and each one has a known cost. We have to select the pipelines in such a way that (1) every plot is connected to the water source, directly or through other plots, and (2) the total cost of the selected pipelines is the minimum possible.

6.2 Existing System

In the usual (manual) method the farmer or the worker decides the pipe layout by guessing or by laying a separate pipe from the source to every plot. This method has some problems:

- A lot of pipe is wasted because nearby plots could share the same line.
- There is no proof that the chosen layout is the cheapest one.
- For a large farm, comparing all possible layouts by hand is very slow and error-prone.

6.3 Proposed System

The proposed system is a C++ program. The user enters the number of points (the water source is point 0 and the plots are 1, 2, 3 ...), the number of possible pipelines and, for every pipeline, the two points it joins and its cost. The program then:

1. checks every input value,
2. sorts the pipelines from the cheapest to the costliest,
3. picks the cheapest pipelines one by one but skips any pipeline that would make a loop,
4. stops when all points are connected, and
5. prints the selected pipelines and the total minimum cost.

6.4 Advantages of the Proposed System

- Gives the cheapest layout, which is guaranteed by Kruskal's algorithm.
- Saves pipe, labour and money.
- Works quickly even if the farm has many plots.
- Tells the user if some plot cannot be connected with the given pipelines.

6.5 Assumptions and Limits

- The cost of a pipeline can be in any unit, for example the length in metres or the money in thousands of rupees.
- A pipeline can carry water in both directions.
- The program accepts at most 20 points and 100 pipelines, and the cost must be a whole number from 1 to 1000.
- Water pressure and the height of the land are not considered, to keep the project simple.

7. Concepts Used

7.1 Graph

A graph is a set of points joined by lines. The points are called vertices and the lines are called edges. When every edge has a number written on it (like cost or distance), the graph is called a weighted graph. In this project:

Graph word	Farm word	Example
Vertex	Water source or a plot	Point 0 = water source, Point 3 = Plot 3
Edge	A possible pipeline	Pipeline between Plot 1 and Plot 2
Weight	Cost of the pipeline	Cost = 1

7.2 Spanning Tree

A spanning tree of a graph connects all the vertices using the smallest possible number of edges, and it has no loop (cycle). If a graph has V vertices, every spanning tree has exactly V - 1 edges. For example, a farm with 6 points (1 source + 5 plots) needs exactly 5 pipelines. Fewer pipes would leave some plot without water, and more pipes would create a loop, which is only a waste of money.

7.3 Minimum Spanning Tree (MST)

A graph can have many spanning trees. The one whose total weight (cost) is the smallest is called the Minimum Spanning Tree. In our project the MST is the cheapest irrigation network.

7.4 Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm, which means that it always takes the best choice available at that moment, here the cheapest pipeline. The steps are:

1. Arrange all pipelines in increasing order of cost.
2. Start with no pipeline selected. Every point is in its own separate group.
3. Take the next cheapest pipeline.
4. If its two end points are in different groups, select the pipeline and join the two groups. If they are already in the same group, skip it, because it would make a loop.
5. Repeat steps 3 and 4 until V - 1 pipelines are selected. The total cost of the selected pipelines is the minimum cost.

7.5 Detecting a Loop using Groups (Union-Find)

To know whether two points are already connected, the program keeps a "group leader" for each point. At the start every point is its own leader. The function findLeader() follows the leaders until it reaches the top leader of a point. If the top leaders of the two end points are the same, the points are already in one group and the pipe is rejected. Otherwise the function joinGroups() makes one leader the leader of the other, so that the two groups become one. This is a simple form of the Union-Find idea.

7.6 Sorting (Selection Sort)

The program has to arrange the pipelines by cost. For this a user-defined selection sort is used: find the smallest cost among the remaining pipelines, put it at the front, and repeat for the next place. It is easy to understand and fast enough for a farm with up to 100 pipelines.

7.7 C++ Concepts Used

- Structure: struct Pipe holds from, to and cost together.
- Arrays: allPipes[], chosen[] and parentOf[] store the data.
- Functions: the program is divided into small user-defined functions.

- Loops and decisions: while, for, if-else for repeating and checking.
- Constants: const int is used for the maximum limits.

7.8 Worked Example

Consider the farm shown in Figure 1 (point 0 is the water source). The numbers on the lines are the costs. The pipelines, after sorting by cost, are examined as shown in the table below.

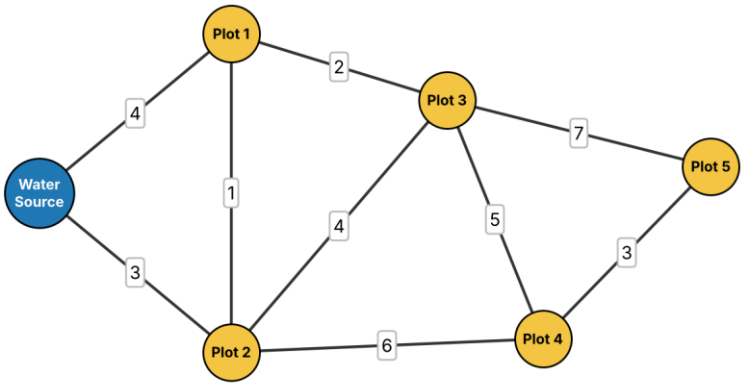


Figure 1: Farm with a water source, five plots and nine possible pipelines

Step	Pipeline	Cost	Same group?	Decision	Total
1	Plot 1 - Plot 2	1	No	Select	1
2	Plot 1 - Plot 3	2	No	Select	3
3	Source - Plot 2	3	No	Select	6
4	Plot 4 - Plot 5	3	No	Select	9
5	Plot 2 - Plot 3	4	Yes (loop)	Reject	9
6	Source - Plot 1	4	Yes (loop)	Reject	9
7	Plot 3 - Plot 4	5	No	Select	14

After step 7 five pipelines (6 - 1 = 5) are selected, so the algorithm stops. The remaining pipelines (cost 6 and 7) are not needed. The minimum total cost is 14.

8. System Design and Program Modules

8.1 Flow of the Program

Figure 2 shows how the program works from start to stop.

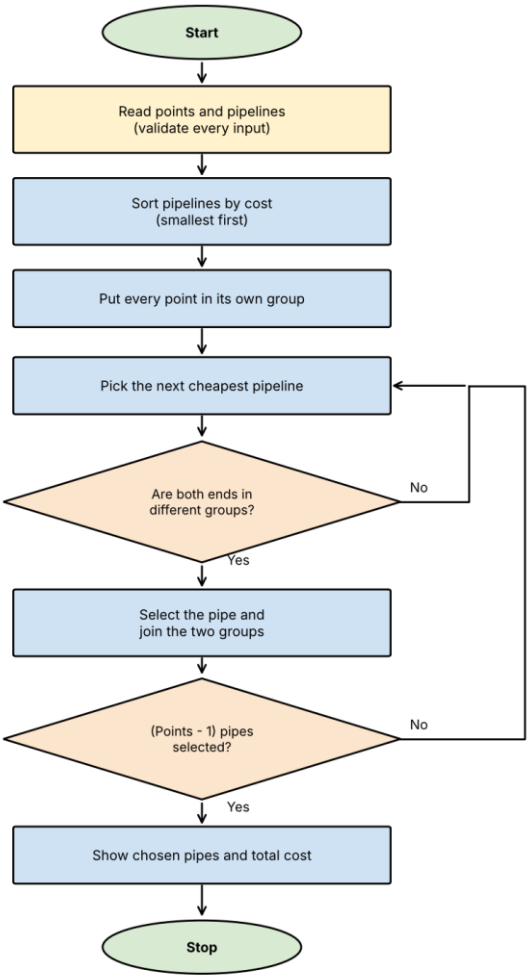


Figure 2: Flowchart of the farm irrigation program

8.2 Data Design

Name	Type	Purpose
Pipe	struct	Stores one pipeline: from, to and cost.
allPipes[100]	Array of Pipe	All possible pipelines entered by the user.
chosen[20]	Array of Pipe	The pipelines selected for the final network.
parentOf[20]	Array of int	Group leader of every point (for loop checking).
totalNodes	int	Water source + number of plots.
pipeCount	int	Number of pipelines entered so far.

8.3 Program Modules

The program is divided into small modules (functions) so that each one does one job.

Function	Job of the function
readInt()	Reads one whole number within a given range. Repeats the question until the input is valid.
pipeExists()	Checks whether a pipeline between two points has already been entered.
readPipes()	Reads all pipelines (two points and cost) and validates each one.
sortPipes()	Sorts the pipelines by cost, smallest first (selection sort).
makeGroups()	Puts every point in its own group at the start.
findLeader()	Finds the group leader of a point.
joinGroups()	Joins two groups; returns false if the points are already in the same group.
buildMinimumSpanningTree()	Runs Kruskal's algorithm and stores the selected pipelines.
printName()	Prints "Water Source" for point 0 and "Plot n" for the others.
showAllPipes()	Shows all entered pipelines after sorting.
showResult()	Shows the selected pipelines, their number and the total cost.
main()	Controls the whole program by calling the above functions in order.

9. Code Implementation

The complete program is given below. Every function is user-defined: no ready-made sorting function, vector, or graph library is used. Only the iostream header is included. Comments are given in the code so that each part can be explained easily.

To compile and run the program using g++:

```
g++ farm_irrigation.cpp -o farm
./farm
```

```
// Farm Irrigation System using Minimum Spanning Tree (Kruskal's Algorithm)
// Author : Tushar Sharma (UID: 26MCC20073)
// Idea   : Node 0 is the Water Source, other nodes are farm plots.
//          Possible pipelines between them have a cost.
//          We find the cheapest set of pipes that connects every plot to water.

#include <iostream>
using namespace std;

const int MAX NODES = 20;   // maximum water source + plots
const int MAX_PIPES = 100;  // maximum possible pipelines
```

```
// A pipeline between two points with a cost
struct Pipe {
    int from;
    int to;
    int cost;
};

Pipe allPipes[MAX_PIPES];   // all possible pipelines entered by user
Pipe chosen[MAX NODES];     // pipelines selected in the final answer
int totalNodes = 0;         // water source + plots
int pipeCount = 0;          // number of pipelines entered
int parentOf[MAX_NODES];    // used by Union-Find to detect cycles

// ---------- Input functions (with validation) ----------

// Reads an integer between low and high. Keeps asking until input is valid.
int readInt(const char *message, int low, int high) {
    int value;
    while (true) {
        cout << message;
        if (!(cin >> value)) {            // user typed letters / symbols
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "  Invalid input! Please enter a number.\n";
        } else if (value < low || value > high) {
            cout << "  Value must be between " << low << " and " << high << ".\n";
        } else {
            return value;
        }
    }
}

// Checks whether a pipeline between a and b was already entered
bool pipeExists(int a, int b) {
    for (int i = 0; i < pipeCount; i++) {
        if ((allPipes[i].from == a && allPipes[i].to == b) ||
            (allPipes[i].from == b && allPipes[i].to == a)) {
            return true;
        }
    }
    return false;
}

// Reads all possible pipelines from the user
void readPipes(int wanted) {
    while (pipeCount < wanted) {
        cout << "\nPipeline " << pipeCount + 1 << ":\n";
        int a = readInt("  Enter first point  (0 = Water Source): ", 0, totalNodes - 1);
        int b = readInt("  Enter second point                   : ", 0, totalNodes - 1);

        if (a == b) {
            cout << "  A pipe cannot join a point to itself. Try again.\n";
            continue;
        }
        if (pipeExists(a, b)) {
            cout << "  This pipeline is already entered. Try again.\n";
            continue;
        }
        int c = readInt("  Enter cost of this pipeline (1-1000)  : ", 1, 1000);

        allPipes[pipeCount].from = a;
        allPipes[pipeCount].to = b;
        allPipes[pipeCount].cost = c;
        pipeCount++;
    }
}

// ---------- Sorting (Selection Sort, written by us) ----------

// Sorts all pipelines from smallest cost to largest cost
void sortPipes() {
```

```
    for (int i = 0; i < pipeCount - 1; i++) {
        int smallest = i;
        for (int j = i + 1; j < pipeCount; j++) {
            if (allPipes[j].cost < allPipes[smallest].cost) {
                smallest = j;
            }
        }
        // swap
        Pipe temp = allPipes[i];
        allPipes[i] = allPipes[smallest];
        allPipes[smallest] = temp;
    }
}

// ---------- Union-Find (to avoid cycles) ----------

// At the start every point is in its own group
void makeGroups() {
    for (int i = 0; i < totalNodes; i++) {
        parentOf[i] = i;
    }
}

// Finds the leader (representative) of the group of point x
int findLeader(int x) {
    while (parentOf[x] != x) {
        x = parentOf[x];
    }
    return x;
}

// Joins the groups of a and b. Returns false if they were already in one group.
bool joinGroups(int a, int b) {
    int leaderA = findLeader(a);
    int leaderB = findLeader(b);
    if (leaderA == leaderB) {
        return false;           // same group -> pipe would make a cycle
    }
    parentOf[leaderA] = leaderB;
    return true;
}

// ---------- Kruskal's Algorithm ----------

// Returns the number of pipes chosen and stores them in chosen[]
int buildMinimumSpanningTree() {
    sortPipes();
    makeGroups();
    int count = 0;
    for (int i = 0; i < pipeCount && count < totalNodes - 1; i++) {
        if (joinGroups(allPipes[i].from, allPipes[i].to)) {
            chosen[count] = allPipes[i];
            count++;
        }
    }
    return count;
}

// ---------- Output functions ----------

// Gives a readable name for a point
void printName(int node) {
    if (node == 0) cout << "Water Source";
    else cout << "Plot " << node;
}

void showAllPipes() {
    cout << "\nAll possible pipelines (sorted by cost):\n";
    cout << "----------------------------------------\n";
    for (int i = 0; i < pipeCount; i++) {
        cout << i + 1 << ". ";
```

```
        printName(allPipes[i].from);
        cout << " <-> ";
        printName(allPipes[i].to);
        cout << "  (cost " << allPipes[i].cost << ")\n";
    }
}

void showResult(int count) {
    cout << "\n===== Minimum Cost Irrigation Network =====\n";
    int total = 0;
    for (int i = 0; i < count; i++) {
        cout << i + 1 << ". ";
        printName(chosen[i].from);
        cout << " <-> ";
        printName(chosen[i].to);
        cout << "  (cost " << chosen[i].cost << ")\n";
        total += chosen[i].cost;
    }
    cout << "-----------------------------------------\n";
    cout << "Pipelines used : " << count << "\n";
    cout << "Total minimum cost : " << total << "\n";
}

// ---------- Main ----------

int main() {
    cout << "=== Farm Irrigation System (Minimum Spanning Tree) ===\n\n";
    cout << "Point 0 is the Water Source. Points 1, 2, 3... are farm plots.\n\n";

    totalNodes = readInt("Enter total points (Water Source + plots, 2-20): ", 2, MAX_NODES);

    int maxPossible = totalNodes * (totalNodes - 1) / 2;
    if (maxPossible > MAX_PIPES) maxPossible = MAX_PIPES;
    int wanted = readInt("Enter number of possible pipelines: ", 1, maxPossible);

    readPipes(wanted);

    int count = buildMinimumSpanningTree();
    showAllPipes();

    if (count == totalNodes - 1) {
        showResult(count);
    } else {
        cout << "\nSome plots cannot be reached with the given pipelines.\n";
        cout << "Please add more possible pipelines and try again.\n";
    }
    return 0;
}
```

10. Sample Input and Output

10.1 Test Case 1

This test uses the farm of Figure 1: 6 points (the water source and 5 plots) and 9 possible pipelines. The input values entered by the user are shown after the colon in each line.

Pipeline	First point	Second point	Cost
1	0	1	4
2	0	2	3
3	1	2	1
4	1	3	2

Pipeline	First point	Second point	Cost
5	2	3	4
6	2	4	6
7	3	4	5
8	3	5	7
9	4	5	3

Program run:

```
=== Farm Irrigation System (Minimum Spanning Tree) ===

Point 0 is the Water Source. Points 1, 2, 3... are farm plots.

Enter total points (Water Source + plots, 2-20): 6
Enter number of possible pipelines: 9

Pipeline 1:
  Enter first point  (0 = Water Source): 0
  Enter second point                   : 1
  Enter cost of this pipeline (1-1000)  : 4

Pipeline 2:
  Enter first point  (0 = Water Source): 0
  Enter second point                   : 2
  Enter cost of this pipeline (1-1000)  : 3

Pipeline 3:
  Enter first point  (0 = Water Source): 1
  Enter second point                   : 2
  Enter cost of this pipeline (1-1000)  : 1

Pipeline 4:
  Enter first point  (0 = Water Source): 1
  Enter second point                   : 3
  Enter cost of this pipeline (1-1000)  : 2

Pipeline 5:
  Enter first point  (0 = Water Source): 2
  Enter second point                   : 3
  Enter cost of this pipeline (1-1000)  : 4

Pipeline 6:
  Enter first point  (0 = Water Source): 2
  Enter second point                   : 4
  Enter cost of this pipeline (1-1000)  : 6

Pipeline 7:
  Enter first point  (0 = Water Source): 3
  Enter second point                   : 4
  Enter cost of this pipeline (1-1000)  : 5

Pipeline 8:
  Enter first point  (0 = Water Source): 3
  Enter second point                   : 5
  Enter cost of this pipeline (1-1000)  : 7

Pipeline 9:
  Enter first point  (0 = Water Source): 4
  Enter second point                   : 5
  Enter cost of this pipeline (1-1000)  : 3
```

```
All possible pipelines (sorted by cost):
----------------------------------------
1. Plot 1 <-> Plot 2   (cost 1)
2. Plot 1 <-> Plot 3   (cost 2)
3. Water Source <-> Plot 2   (cost 3)
4. Plot 4 <-> Plot 5   (cost 3)
5. Plot 2 <-> Plot 3   (cost 4)
6. Water Source <-> Plot 1   (cost 4)
7. Plot 3 <-> Plot 4   (cost 5)
8. Plot 2 <-> Plot 4   (cost 6)
9. Plot 3 <-> Plot 5   (cost 7)

===== Minimum Cost Irrigation Network =====
1. Plot 1 <-> Plot 2   (cost 1)
2. Plot 1 <-> Plot 3   (cost 2)
3. Water Source <-> Plot 2   (cost 3)
4. Plot 4 <-> Plot 5   (cost 3)
5. Plot 3 <-> Plot 4   (cost 5)
----------------------------------------
Pipelines used : 5
Total minimum cost : 14
```

The selected pipelines are shown in Figure 3 (green lines). The dashed grey lines are the pipelines that are not used.

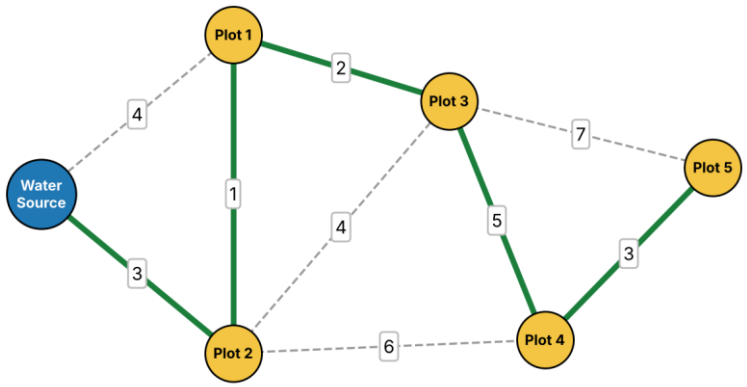


Figure 3: Minimum spanning tree (green) of the farm, total cost = 14

10.2 Test Case 2

A smaller farm with 4 points and 4 possible pipelines. The costly pipeline between the water source and Plot 3 (cost 10) is not needed because Plot 3 can be reached through Plot 2.

```
=== Farm Irrigation System (Minimum Spanning Tree) ===

Point 0 is the Water Source. Points 1, 2, 3... are farm plots.

Enter total points (Water Source + plots, 2-20): 4
Enter number of possible pipelines: 4

Pipeline 1:
  Enter first point  (0 = Water Source): 0
  Enter second point                   : 1
  Enter cost of this pipeline (1-1000)  : 5

Pipeline 2:
  Enter first point  (0 = Water Source): 1
  Enter second point                   : 2
  Enter cost of this pipeline (1-1000)  : 3
```

```
Pipeline 3:
  Enter first point  (0 = Water Source): 2
  Enter second point                   : 3
  Enter cost of this pipeline (1-1000)  : 4

Pipeline 4:
  Enter first point  (0 = Water Source): 0
  Enter second point                   : 3
  Enter cost of this pipeline (1-1000)  : 10

All possible pipelines (sorted by cost):
---------------------------------------
1. Plot 1 <-> Plot 2  (cost 3)
2. Plot 2 <-> Plot 3  (cost 4)
3. Water Source <-> Plot 1  (cost 5)
4. Water Source <-> Plot 3  (cost 10)

===== Minimum Cost Irrigation Network =====
1. Plot 1 <-> Plot 2  (cost 3)
2. Plot 2 <-> Plot 3  (cost 4)
3. Water Source <-> Plot 1  (cost 5)
-------------------------------------------
Pipelines used : 3
Total minimum cost : 12
```

Result: 3 pipelines are used (4 - 1 = 3) and the minimum total cost is 12.

11. Input Validation

A program should not give wrong results or stop when the user types a wrong value. This program checks every input before using it.

Check	What the program does	Message shown
Not a number (for example letters)	readInt() clears the error, throws away the wrong text and asks again.	Invalid input! Please enter a number.
Number out of range	Points must be 2-20, pipelines 1 up to the possible maximum, point numbers 0 to (points - 1), cost 1-1000.	Value must be between ... and ...
Pipe joins a point to itself	The pipeline is rejected and the user enters it again.	A pipe cannot join a point to itself.
Duplicate pipeline	pipeExists() checks both directions (1-0 is the same as 0-1).	This pipeline is already entered.
Network not connected	If fewer than (points - 1) pipes can be selected, no result is printed.	Some plots cannot be reached ...

Sample run showing the validation messages:

```
=== Farm Irrigation System (Minimum Spanning Tree) ===

Point 0 is the Water Source. Points 1, 2, 3... are farm plots.

Enter total points (Water Source + plots, 2-20): abc
  Invalid input! Please enter a number.
Enter total points (Water Source + plots, 2-20): 1
  Value must be between 2 and 20.
Enter total points (Water Source + plots, 2-20): 6
```

```
Enter number of possible pipelines: xyz
  Invalid input! Please enter a number.
Enter number of possible pipelines: 50
  Value must be between 1 and 15.
Enter number of possible pipelines: 3


Pipeline 1:
  Enter first point  (0 = Water Source): 2
  Enter second point                   : 2
  A pipe cannot join a point to itself. Try again.


Pipeline 1:
  Enter first point  (0 = Water Source): 0
  Enter second point                   : 9
  Value must be between 0 and 5.
  Enter second point                   : 1
  Enter cost of this pipeline (1-1000)  : 2


Pipeline 2:
  Enter first point  (0 = Water Source): 1
  Enter second point                   : 0
  This pipeline is already entered. Try again.


Pipeline 2:
  Enter first point  (0 = Water Source): -5
  Value must be between 0 and 5.
  Enter first point  (0 = Water Source): 0
  Enter second point                   : 5
  Enter cost of this pipeline (1-1000)  : 2


Pipeline 3:
  Enter first point  (0 = Water Source): 2
  Enter second point                   : 1
  Enter cost of this pipeline (1-1000)  : 1


All possible pipelines (sorted by cost):
----------------------------------------
1. Plot 2 <-> Plot 1  (cost 1)
2. Water Source <-> Plot 5  (cost 2)
3. Water Source <-> Plot 1  (cost 2)


Some plots cannot be reached with the given pipelines.
Please add more possible pipelines and try again.
```

In this run the user first typed letters (abc) and then values outside the allowed range (1, 50, 9, -5). A self-loop (2 to 2) and a duplicate pipe (1 to 0, same as 0 to 1) were also rejected. Finally, since Plots 2, 3 and 4 could not be reached with the three valid pipelines, the program reported that the network is not connected instead of printing a wrong answer.

12. Features and Functionalities

- **Minimum cost network:** finds the cheapest way to connect the water source to all plots.
- **Easy input:** the user only has to type numbers when asked, so no training is needed.
- **Complete input validation:** wrong type, wrong range, self-loop and duplicate pipelines are handled.
- **Loop-free layout:** a pipeline that would create a useless loop is automatically skipped.

- **Sorted display:** all pipelines are shown from cheapest to costliest, and the selected ones are listed with the total cost.
- **Connectivity check:** tells the user when some plot cannot be connected.
- **User-defined functions:** the program is divided into small modules that are easy to read, test and explain.
- **No special libraries:** only iostream is used, so it runs on any C++ compiler.
- **Readable output:** points are shown as "Water Source" and "Plot n" instead of only numbers.

13. Future Enhancements

- **Plot names:** allow the user to give real names to the plots instead of numbers.
- **Cost from distance:** ask for the distance in metres and the price per metre, and calculate the cost automatically.
- **Save and load data:** store the farm details and the result in a file so that they need not be typed again.
- **Graphical output:** draw the farm and the selected pipelines on the screen or on a map.
- **Faster algorithm:** use a faster sorting method or Prim's algorithm for very large farms, and compare both.
- **More than one water source:** support farms that have several wells or tanks.
- **Water pressure and land height:** consider these real-world factors while choosing pipelines.
- **Mobile / web version:** make an app so that farmers can plan the network from a phone.
- **Smart farming:** connect the plan with sensors and automatic valves (IoT) for controlled watering.

14. Conclusion

In this project a farm irrigation network was planned with the help of a Minimum Spanning Tree. The farm was represented as a graph, and Kruskal's algorithm was implemented in C++ to select the cheapest set of pipelines that connects all plots to the water source. The program uses only simple ideas, namely structures, arrays, functions, loops, a user-written sort and a simple group method for loop checking, so it is easy to understand and to explain.

The tests show that the program gives the correct minimum cost (14 for the main example and 12 for the smaller example) and handles wrong inputs safely. The project shows how a classroom algorithm can be used to solve a real problem like saving money and material in farming. It can be extended further with the enhancements listed in this report.

15. References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.

[2] E. Horowitz, S. Sahni and S. Rajasekaran, Fundamentals of Computer Algorithms, Universities Press.

[3] E. Balagurusamy, Object Oriented Programming with C++, McGraw Hill Education.

[4] Kruskal's Algorithm, Programiz. https://www.programiz.com/dsa/kruskal-algorithm

[5] Kruskal's Minimum Spanning Tree (MST) Algorithm, GeeksforGeeks. https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/

[6] C++ reference, cppreference.com. https://en.cppreference.com